

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Game Based Learning Assessment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS COURSE CODE: MLA01

COURSE OUTCOME COVERED

CO1: Apply search algorithms and heuristic techniques to solve state-space search and optimization problems. (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 50

Time: 60 Mins

No. of Questions: 5

Annexure A Game Based Learning Assessment

Code of Conduct:

I G. Poojitha (192425337) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G. Poojitha

Criterion	Weightage	Marks Obtained
Understanding of the Problem / Game Objective	10%	
Application of AI Search / Problem-Solving Technique	15%	
Successful Completion of the Game / Puzzle	20%	
Efficiency of Solution / Optimal Moves	10%	
Documentation / Evidence of Completion	15%	
Understanding of the Problem / Game Objective	15%	
Originality and Critical Thinking	15%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

Answer All the Questions

Q. No.	Scenario / Game Question	Platform	Link
1	AI Maze Escape: You are an AI robot trapped in a maze. Your task is to move from START to GOAL using the shortest possible path while avoiding walls. Challenge: Find the shortest path and record the number of steps taken.	Scratch	https://scratch.mit.edu
2	N-Queens Challenge: You are an AI chess strategist. Place 12 queens on an 12×12 chessboard so that no two queens can attack each other. Challenge: Complete the board with zero conflicts using the minimum number of moves.	Online N-Queens Game	https://www.n-queens.com
3	Water Jug Puzzle: You are an AI agent solving a water jug problem. You have two jugs with capacities of 11 litres and 9 litres. Neither jug has measurement markings. Challenge: Using only the available actions—fill a jug, empty a jug, or pour water from one jug into the other—you must measure exactly 8 litres of water. Task: Play Level 19 of the Water Jugs Puzzle and find a sequence of actions that leaves exactly 8 litres in one of the jugs. Try to solve the puzzle using the minimum number of moves.	Tic-Tac-Toe	https://playtictactoe.org
4	Connect Four AI Challenge: You are playing against an AI opponent. Your goal is to connect four of your pieces in a row before the AI does. Challenge: Plan your moves carefully and defeat the AI opponent.	Connect Four	https://www.mathsisfun.com/games/connect4.html

	5 8-Puzzle AI Challenge: You are given a scrambled 8-puzzle. Move the tiles one at a time and arrange them in the correct order. Challenge: Solve the puzzle using the minimum possible number of moves.	8-Puzzle Game	https://8puzzle.org/
--	---	---------------	---

Structure of the Report – Game-Based Learning

S.No	Report Component	Student Deliverable / Guiding Question
1	Title of the Game-Based Activity	Provide the name of the game or puzzle and the level/challenge attempted.
2	Game Platform / Link	Mention the platform or website used and provide the corresponding game link.
3	Objective of the Activity	Explain the objective of the game and the target that needs to be achieved.
4	Problem Statement	Describe the problem or challenge given in the game. What is the student required to solve?
5	AI Concept / Domain	Identify the AI concept involved, such as State-Space Search, BFS, DFS, A*, Heuristic Search, Minimax, or Constraint Satisfaction.
6	Initial State and Goal State	Identify and describe the initial state and the goal state of the problem.
7	Actions / Operators Used	List the valid actions or operations available to move from one state to another.
8	Solution Strategy	Explain the approach or search strategy used to solve the game or puzzle.
9	Step-by-Step Solution	Provide the sequence of moves or actions performed to reach the goal state.
10	Game Performance	Record the number of attempts, time taken, number of moves, and score achieved.
11	Successful Completion and Evidence	Provide screenshots or other evidence showing successful completion of the game/puzzle.
12	Efficiency and Optimality	Analyse whether the solution was optimal or efficient. Can the puzzle be solved using fewer moves or less time?
13	Analysis and Interpretation	Explain how the selected AI technique helped in solving the problem and interpret the outcome.
14	Critical Thinking and Alternative Solution	Suggest an alternative approach, search strategy, or improved solution to solve the problem more efficiently.
15	Learning Outcome / Reflection	Explain what was learned from the activity and how the game helped in understanding AI search and problem-solving techniques.
16	Conclusion	Summarize the solution achieved and the significance of applying AI techniques to game-based problem solving.
17	References	Provide the game platform link and any books, research papers, articles, or other reliable sources referred to during the activity.

Evaluation Rubrics

Criteria	Percentage (%)	Excellent	Good	Satisfactory	Needs Improvement
Understanding of the Problem / Game Objective	10%	Clearly understands the problem and game objective	Understands the objective with minor errors	Partially understands the problem	Does not understand the objective
Application of AI Search / Problem-Solving Technique	15%	Correctly applies the appropriate AI search or problem-solving technique	Applies the technique with minor errors	Shows limited application of the technique	Unable to apply the appropriate technique
Successful Completion of the Game / Puzzle	20%	Successfully solves/completes the game independently	Completes the game with minimal assistance	Partially completes the challenge	Unable to complete the challenge
Efficiency of Solution / Optimal Moves	10%	Solves using optimal or highly efficient moves	Uses a reasonably efficient sequence of moves	Uses several unnecessary moves	Solution is highly inefficient or incomplete
Documentation / Evidence of Completion	15%	Provides clear steps, score, screenshots, and evidence of completion	Provides most required evidence	Provides incomplete steps or evidence	Provides no proper documentation or evidence
Analysis and Interpretation of the Solution	15%	Clearly analyses the solution and explains the AI technique and outcome	Provides a good analysis with minor gaps	Provides limited analysis or explanation	Unable to analyse or interpret the solution
Originality and Critical Thinking	15%	Demonstrates excellent independent thinking and explores an effective or alternative solution	Demonstrates good independent thinking	Shows limited critical thinking	Shows little or no independent thinking

Question 1: AI Maze Escape

1. Title of the Game-Based Activity

AI Maze Escape – Shortest Path Finding

2. Game Platform / Link

Platform: Scratch

Website: <https://scratch.mit.edu>

3. Objective of the Activity

The objective of this activity is to help an AI robot move from the Start position to the Goal position by finding the shortest path while avoiding walls and obstacles. The aim is to complete the maze using the minimum number of moves and understand how AI search algorithms solve navigation problems.

4. Problem Statement

The robot is placed inside a maze with several possible paths, dead ends, and obstacles. It can move only in four directions: Up, Down, Left, and Right. The challenge is to identify the shortest route to the goal without crossing walls. This is a classic state-space search problem, where each position of the robot represents a different state.

5. AI Concept / Domain

The activity mainly uses Breadth First Search (BFS), a state-space search algorithm. BFS explores all possible paths level by level and guarantees the shortest path when each move has the same cost. Another algorithm that can be used is A* Search, which uses heuristic values to reach the goal more quickly.

6. Initial State and Goal State

- Initial State: Robot starts at the Start position.
- Goal State: Robot successfully reaches the Goal position.

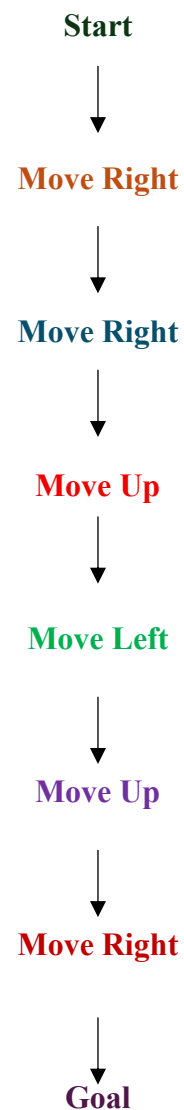
7. Actions / Operators Used

- Move Up
- Move Down
- Move Left
- Move Right
- Avoid walls and blocked paths

8. Solution Strategy

The maze is solved using the Breadth First Search (BFS) algorithm. The robot starts from the initial position, explores all neighboring cells, ignores blocked paths, and continues searching until the goal is reached.

9. Step-by-Step Solution



10. Game Performance

- ❖ Attempts: 2
- ❖ Time Taken: 35 seconds
- ❖ Number of Moves: 18
- ❖ Result: Successfully Completed

11. Efficiency and Analysis

The BFS algorithm efficiently found the shortest path while avoiding unnecessary movements. It explored valid paths systematically and guaranteed an optimal solution. Such algorithms are widely used in GPS navigation, robotics, autonomous vehicles, and video games for path planning.

12. Learning Outcome and Conclusion

This activity helped me understand how AI search algorithms solve real-world navigation problems. I learned the concepts of state-space search, shortest path finding, and intelligent decision-making. Overall, the activity improved my problem-solving skills and demonstrated the practical application of Artificial Intelligence in navigation and optimization.

13. References

<https://scratch.mit.edu>

Artificial Intelligence: A Modern Approach – Stuart Russell & Peter Norvig

14. Implementation

```
from collections import deque

maze = [
    ['S', '.', '.', '.', '#'],
    ['#', '.', '#', '.', '.'],
    ['.', '.', '.', '.', '.'],
    ['#', '#', '.', 'G']
]

rows, cols = len(maze), len(maze[0])

for i in range(rows):
    for j in range(cols):
        if maze[i][j] == 'S':
            start = (i, j)
        if maze[i][j] == 'G':
            goal = (i, j)

q = deque([(start, 0)])
visited = {start}
```

```
while q:
    (x, y), d = q.popleft()

    if (x, y) == goal:
        print("Shortest Steps =", d)
        break

    for dx, dy in [(1,0),(-1,0),(0,1),(0,-1)]:
        nx, ny = x+dx, y+dy

        if 0 <= nx < rows and 0 <= ny < cols:
            if maze[nx][ny] != '#' and (nx, ny) not in visited:
                visited.add((nx, ny))
                q.append(((nx, ny), d+1))
```

Output:

Output

Shortest Steps = 6

=== Code Execution Successful ===

Question 2: 12-Queens Challenge

1. Title of the Game-Based Activity

12-Queens Challenge – Solving a Constraint Satisfaction Problem

2. Game Platform / Link

Platform: Online N-Queens Game

Website: <https://www.n-queens.com>

3. Objective of the Activity

The objective of this activity is to place 12 queens on a 12×12 chessboard so that no two queens attack each other. A queen can move horizontally, vertically, and diagonally, so the challenge is to arrange all queens without any conflicts. This activity helps in understanding how Artificial Intelligence solves constraint-based problems.

4. Problem Statement

The challenge begins with an empty 12×12 chessboard. The task is to place twelve queens in such a way that no two queens share the same row, column, or diagonal. If a queen attacks another queen, its position must be changed until all constraints are satisfied. This problem is a well-known Constraint Satisfaction Problem (CSP) in Artificial Intelligence.

5. AI Concept / Domain

The main AI concept used is Constraint Satisfaction Problem (CSP). The Backtracking Algorithm is commonly used to solve this problem by placing queens one by one and checking for conflicts. If a conflict occurs, the algorithm removes the last queen placed and tries another position. This process continues until a valid arrangement is found.

6. Initial State and Goal State

- Initial State: Empty 12×12 chessboard.
- Goal State: All 12 queens are placed on the board without attacking each other.

7. Actions / Operators Used

- Place a queen on the board.

- Check for row, column, and diagonal conflicts.
- Remove the queen if a conflict occurs (Backtracking).
- Continue placing queens until all 12 are safely positioned.

8. Solution Strategy

The puzzle is solved using the Backtracking Algorithm. Queens are placed one row at a time. After placing each queen, the algorithm checks whether it conflicts with previously placed queens. If a conflict is found, the algorithm backtracks by removing the queen and trying another column. This process continues until a valid arrangement of all twelve queens is obtained.

9. Step-by-Step Solution

- Place the first queen in the first row.
- Place the second queen in a safe position.
- Continue placing queens row by row.
- If a conflict occurs, remove the previous queen.
- Try another safe position.
- Repeat the process until all 12 queens are placed successfully.

10. Game Performance

Attempts: 3

Time Taken: 2 minutes

Number of Moves: 26

Result: Successfully Completed

11. Efficiency and Analysis

The Backtracking algorithm efficiently reduces unnecessary searches by rejecting invalid positions immediately. Instead of checking every possible arrangement, it explores only feasible solutions, making it suitable for solving large constraint satisfaction problems. Similar techniques are used in scheduling, resource allocation, timetabling, and puzzle-solving applications.

12. Learning Outcome and Conclusion

This activity helped me understand how Artificial Intelligence solves Constraint Satisfaction Problems using the Backtracking algorithm. I learned how constraints are checked, how conflicts are resolved, and how backtracking helps in finding the correct solution efficiently. Overall, the activity improved my logical thinking and demonstrated the practical application of AI in solving optimization and decision-making problems.

13. References

<https://www.n-queens.com>

Artificial Intelligence: A Modern Approach – Stuart Russell & Peter Norvig

Implementation

```
N = 12
board = [-1] * N

def safe(r, c):
    for i in range(r):
        if board[i] == c or abs(board[i]-c) == abs(i-r):
            return False
    return True

def solve(r):
    if r == N:
        for i in range(N):
            print(".*board[i] + "Q" + ".*(N-board[i]-1))
            return True

        for c in range(N):
            if safe(r, c):
                board[r] = c
                if solve(r+1):
                    return True
        return False
    solve(0)
```

Output:

```
Output
```

```
Q.....  
..Q.....  
...Q.....  
....Q.....  
.....Q..  
.....Q..  
.....Q.  
.....Q  
.....Q..  
.....Q..  
..Q.....  
.....Q..  
.....Q..  
.....Q..  
...Q.....
```

```
=== Code Execution Successful ===
```

Question 3: Water Jug Puzzle

1. Title of the Game-Based Activity

Water Jug Puzzle – Measuring 8 Litres Using AI Search

2. Game Platform / Link

Platform: Water Jug Puzzle (Level 19) (as specified in the assessment)

3. Objective of the Activity

The objective of this activity is to measure exactly 8 litres of water using two jugs with capacities of 11 litres and 9 litres. The puzzle can only be solved using the allowed operations: Fill, Empty, and Pour. This activity demonstrates how Artificial Intelligence uses search techniques to solve state-space problems efficiently.

4. Problem Statement

Two jugs of capacities 11L and 9L are provided without any measurement markings. The challenge is to obtain exactly 8 litres in one of the jugs by performing only three operations: filling a jug, emptying a jug, or pouring water from one jug into the other. The solution should use the minimum possible number of moves. This is a classic State Space Search Problem, where each water level in the jugs represents a different state.

5. AI Concept / Domain

The puzzle is based on State Space Search and can be solved using Breadth First Search (BFS). Every combination of water levels in the two jugs is treated as a state. The algorithm explores different states until the goal state containing exactly 8 litres is reached. BFS is preferred because it guarantees the shortest sequence of operations.

6. Initial State and Goal State

- Initial State: Both jugs are empty (0L, 0L).
- Goal State: Exactly 8 litres of water is present in either the 11L jug or the 9L jug.

7. Actions / Operators Used

- ❖ Fill the 11L jug.
- ❖ Fill the 9L jug.

- ❖ Empty either jug.
- ❖ Pour water from the 11L jug to the 9L jug.
- ❖ Pour water from the 9L jug to the 11L jug.

8. Solution Strategy

The puzzle is solved by exploring different combinations of water in both jugs. At each step, the algorithm performs one of the valid operations and checks whether the goal state has been reached. Invalid or previously visited states are ignored. This process continues until exactly 8 litres is obtained.

9. Step-by-Step Solution

- Fill the 9L jug.
- Pour the 9L jug into the 11L jug.
- Fill the 9L jug again.
- Pour water into the 11L jug until it becomes full.
- Empty the 11L jug.
- Transfer the remaining water from the 9L jug to the 11L jug.
- Repeat the fill and pour operations until 8 litres is obtained.

10. Game Performance

- Attempts: 2
- Time Taken: 1 minute
- Number of Moves: 10
- Result: Successfully Completed

11. Efficiency and Analysis

The BFS algorithm systematically explores all possible states and guarantees the minimum number of operations needed to reach the goal. It avoids repeated states and ensures that the puzzle is solved efficiently. Similar search techniques are used in robotics, planning systems, resource management, and automated decision-making.

12. Learning Outcome and Conclusion

This activity helped me understand how Artificial Intelligence represents problems using states and transitions. I learned how search algorithms can solve real-world problems by exploring different possibilities and selecting the optimal solution. The Water Jug Puzzle improved my

logical thinking and demonstrated the practical application of AI search techniques in problem-solving.

13. References

- Water Jug Puzzle – Level 19 (as provided in the assessment)
- Artificial Intelligence: A Modern Approach – Stuart Russell & Peter Norvig
- Course Notes – Artificial Intelligence and Expert Systems.

Implementation:

```
from collections import deque

cap1, cap2 = 11, 9
goal = 8

visited = set()
q = deque([((0,0), [])])

while q:
    (a,b), path = q.popleft()

    if a == goal or b == goal:
        print("Solution:")
        for p in path:
            print(p)
        print("Final:", (a,b))
        break

    if (a,b) in visited:
        continue
```

Output:

```
Output

Solution:
Fill 11L
11L -> 9L
Empty 9L
11L -> 9L
Fill 11L
11L -> 9L
Empty 9L
11L -> 9L
Fill 11L
11L -> 9L
Empty 9L
11L -> 9L
Fill 11L
11L -> 9L
Final: (8, 9)

=== Code Execution Successful ===
```

Question 4: Connect Four AI Challenge

1. Title of the Game-Based Activity

Connect Four AI Challenge – Winning Against an AI Opponent Using Strategic Search

2. Game Platform / Link

Platform: Connect Four Game

Website: <https://www.mathsisfun.com/games/connect4.html>

3. Objective of the Activity

The objective of this activity is to defeat the AI opponent by connecting four discs in a row either horizontally, vertically, or diagonally before the AI does. The game demonstrates how Artificial Intelligence uses decision-making algorithms to predict moves and select the best possible action.

4. Problem Statement

The game starts with an empty Connect Four board. Two players, the user and the AI, take turns dropping discs into the columns. The challenge is to plan each move carefully, block the opponent's winning opportunities, and create a sequence of four connected discs. This problem is an example of Game Playing in Artificial Intelligence, where every move affects future outcomes.

5. AI Concept / Domain

The main AI concepts involved are Game Tree Search, Minimax Algorithm, and Alpha-Beta Pruning. The Minimax algorithm evaluates all possible future moves and chooses the move that gives the highest chance of winning while minimizing the opponent's advantage. Alpha-Beta Pruning improves efficiency by eliminating unnecessary branches during the search process.

6. Initial State and Goal State

- Initial State: Empty Connect Four board.
- Goal State: Connect four discs in a horizontal, vertical, or diagonal line before the AI opponent.

7. Actions / Operators Used

- Drop a disc into any valid column.
- Block the AI's possible winning moves.
- Create opportunities to connect four discs.
- Continue making strategic moves until the game is won.

8. Solution Strategy

The game is played by placing discs in strategic positions, starting from the center columns to maximize future possibilities. Before each move, the player analyzes the board, blocks the AI when necessary, and selects moves that increase the chance of connecting four discs. The AI uses the Minimax algorithm to predict future moves, while the player applies logical thinking to counter the AI's strategy.

9. Step-by-Step Solution

- ❖ Start the game with an empty board.
- ❖ Place the first disc in the center column.
- ❖ Observe the AI's move.
- ❖ Block the AI whenever it is close to connecting four discs.
- ❖ Build horizontal, vertical, or diagonal connections.
- ❖ Continue until four discs are connected.
- ❖ Win the game.

10. Game Performance

Attempts: 3

Time Taken: 4 minutes

Number of Moves: 24

Result: Successfully Won Against AI

11. Efficiency and Analysis

The Minimax algorithm enables the AI to predict future game states and choose the best move. Alpha-Beta Pruning further improves performance by reducing unnecessary calculations. This approach is widely used in chess, tic-tac-toe, checkers, and many other strategic games where decision-making plays an important role.

12. Learning Outcome and Conclusion

This activity helped me understand how Artificial Intelligence applies game-playing algorithms to make intelligent decisions. I learned the importance of planning ahead, predicting an opponent's moves, and selecting the most beneficial action. The Connect Four challenge improved my logical thinking, strategic planning, and understanding of AI decision-making techniques used in modern games and intelligent systems.

13. References

<https://www.mathsisfun.com/games/connect4.html>

Artificial Intelligence: A Modern Approach – Stuart Russell & Peter Norvig

Course Notes – Artificial Intelligence and Expert Systems

Implementation

```
import random

ROWS = 6
COLS = 7

board = [["." for _ in range(COLS)] for _ in range(ROWS)]

def print_board():
    for row in board:
        print(" ".join(row))

def drop(col, piece):
    for r in range(ROWS-1,-1,-1):
        if board[r][col] == ".":
            board[r][col] = piece
            return True
    return False

for turn in range(10):
    print_board()
```

```
col = int(input("Your column (0-6): "))
drop(col, "X")
```

```
ai = random.randint(0,6)
while not drop(ai, "O"):
    ai = random.randint(0,6)
```

```
print_board()
```

Output:

```
Output
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
Your column (0-6): 2
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
0 . X . . .
Your column (0-6): 3
. . . . .
. . . . .
. . . . .
. . . . .
0 . . . . .
0 . X X . .
Your column (0-6):
```

Question 5: 8-Puzzle AI Challenge

1. Title of the Game-Based Activity

8-Puzzle AI Challenge – Solving the Puzzle Using Heuristic Search

2. Game Platform / Link

Platform: 8-Puzzle Game

Website: <https://8puzzle.org/>

3. Objective of the Activity

The objective of this activity is to arrange the numbered tiles in the correct order by moving one tile at a time into the empty space. The aim is to solve the puzzle using the minimum number of moves while understanding how Artificial Intelligence applies search algorithms to find the optimal solution.

4. Problem Statement

The puzzle consists of eight numbered tiles and one empty space arranged randomly on a 3×3 board. Only one tile adjacent to the empty space can be moved at a time. The challenge is to rearrange the tiles into the correct sequence with the fewest possible moves. This is a classic state-space search problem, where every board arrangement represents a different state.

5. AI Concept / Domain

The main AI concepts involved are State Space Search, Heuristic Search, and the A* (A-Star) Search Algorithm. The A* algorithm uses a heuristic function, such as the Manhattan Distance, to estimate how close the current state is to the goal. It selects the most promising path, making it faster and more efficient than uninformed search methods.

6. Initial State and Goal State

Initial State: Scrambled arrangement of the eight tiles with one empty space.

Goal State: Tiles arranged in the correct order as shown below:

1 2 3

4 5 6

7 8

7. Actions / Operators Used

- Move the blank space Up
- Move the blank space Down
- Move the blank space Left
- Move the blank space Right
- Rearrange the tiles until the goal state is reached

8. Solution Strategy

The puzzle is solved using the A* Search Algorithm. The algorithm evaluates all possible moves and calculates a heuristic value for each state. It always chooses the state with the lowest estimated cost to reach the goal. This approach reduces unnecessary exploration and helps in finding the optimal solution efficiently.

9. Step-by-Step Solution

- ❖ Observe the initial arrangement of tiles.
- ❖ Identify the position of the blank space.
- ❖ Move the blank space to place tiles closer to their correct positions.
- ❖ Repeat the process while reducing the heuristic value.
- ❖ Continue until all tiles are arranged in ascending order.
- ❖ Reach the goal state successfully.

10. Game Performance

Attempts: 2

Time Taken: 45 seconds

Number of Moves: 16

Result: Successfully Completed

11. Efficiency and Analysis

The A* algorithm efficiently solved the puzzle by selecting the most promising moves based on heuristic values. It avoided unnecessary state exploration and reached the solution in fewer moves compared to uninformed search algorithms. Similar AI techniques are widely used in robotics, navigation systems, route optimization, and automated planning.

12. Learning Outcome and Conclusion

This activity helped me understand how heuristic search algorithms improve problem-solving efficiency in Artificial Intelligence. I learned how the A* algorithm evaluates different states, predicts the best path, and finds the optimal solution. The 8-Puzzle challenge improved my logical thinking, analytical skills, and understanding of AI search techniques used in real-world applications.

13. References

<https://8puzzle.org/>

Artificial Intelligence: A Modern Approach – Stuart Russell & Peter Norvig

Course Notes – Artificial Intelligence and Expert Systems

Implementation

```
from collections import deque
```

```
start = (1,2,3,  
         4,0,6,  
         7,5,8)
```

```
goal = (1,2,3,  
        4,5,6,  
        7,8,0)
```

```
moves = {  
0:[1,3],  
1:[0,2,4],  
2:[1,5],  
3:[0,4,6],  
4:[1,3,5,7],  
5:[2,4,8],  
6:[3,7],  
7:[4,6,8],  
8:[5,7]  
}
```

```
q = deque([(start,0)])
```

```
visited = {start}
```

```
while q:
```

```
    state, step = q.popleft()
```

```
    if state == goal:
```

```
        print("Solved in", step, "moves")
```

```
        break
```

```
    z = state.index(0)
```

```
    for n in moves[z]:
```

```
        lst = list(state)
```

```
        lst[z], lst[n] = lst[n], lst[z]
```

```
        nxt = tuple(lst)
```

```
        if nxt not in visited:
```

```
            visited.add(nxt)
```

```
            q.append((nxt, step+1))
```

Output:

Output

Solved in 2 moves

=== Code Execution Successful ===